# Modulo 2: Aplicaciones distribuidas

## Tema 1: Microservicios

Las arquitecturas tradicionales (C/S, o P2P) complican el desarrollo de aplicaciones altamente masivas

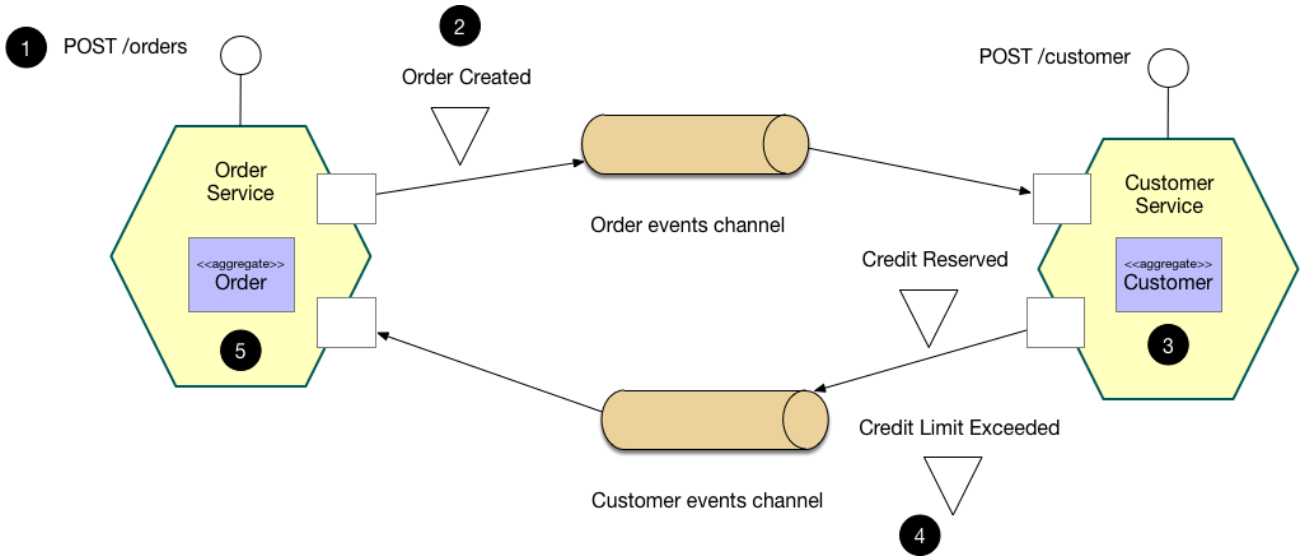
### Arquitectura y metodologia de desarrollo modernas:

- **Escenario**:
  - Equipo de desarrolladores trabajando en la aplicacion
  - Nuevos miembro deben ser productivos rapidamente
  - Aplicacion debe ser facil de entender y mantener
  - Aplicacion debe poder atender a miles de usuarios concurrentes
  - Se desea aprovechar las tecnologias emergentes
- **Solucion**:
  - Definir una arquitectura que estructure la aplicacion como un conjunto de servicios que colaboran entre si (Microservicios)
- **Ventajas**:
  - Fácilmente mantenibles y testables Permite desarrollos y despliegues rápidos y frecuentes
  - Bajo acoplamiento (dependencia) con otros servicios
  - Desplegable de forma independiente Cada equipo puede desplegar su servicio sin tener que coordinarse con otros equipos, normalmente utilizando tecnología de contenedores (Docker)
  - Permite equipos de desarrollo pequeños Aumenta la productividad al evitar la alta carga de trabajo debida a la comunicación y coordinación en los equipos grandes
- **Caracteristicas**:
  - Independencia en el desarrollo (Se comunican mediante protocolos sincronos o asincronos)
  - Independencia en el despliegue y arquitectura (Cada microservicio suele ejecutarse en un contenedor dedicado)
  - Independencia en el modelo de datos (Cada servicio tiene su propia base de datos. La coherencia de los datos se mantiene mediante el patron Saga)

### Patron Saga

Implementar cada transaccion como una saga (Secuencia de transacciones locales coordinadas por eventos):

- Cada transaccion actualiza su base de datos y genera un evento para disparar la siguiente transaccion
- Si una transaccion falla, la saga ejecuta una serie de operaciones para deshacer los cambios




1. El `Order Service` recibe la peticion al API `POST /orders` y crea un **Order** con estado **PENDING**
2. Se emite un evento (mensaje) `Order Created`, y se introduce en la cola correspondiente
3. El manejador del **Servicio de Clientes** procesa el mensaje e intenta reservar el numero de entradas solicitadas
4. Se emite un evento (mensaje) indicando el resultado (exito o fracaso)
5. El servicio de **Servicio de Pedidos** recibe el mensaje y acepta o rechaza el pedido, ajustando el estaod del mismo (**PENDING** > **ACCEPTED** o **REJECTED**)

# Tema 2: Colas de Mensajes

Sistema de comunicacion **asincrono** entre procesos Ayuda a desacoplar sus ritmos de trabajo y absorber picos (tanto de proceso como de envio)

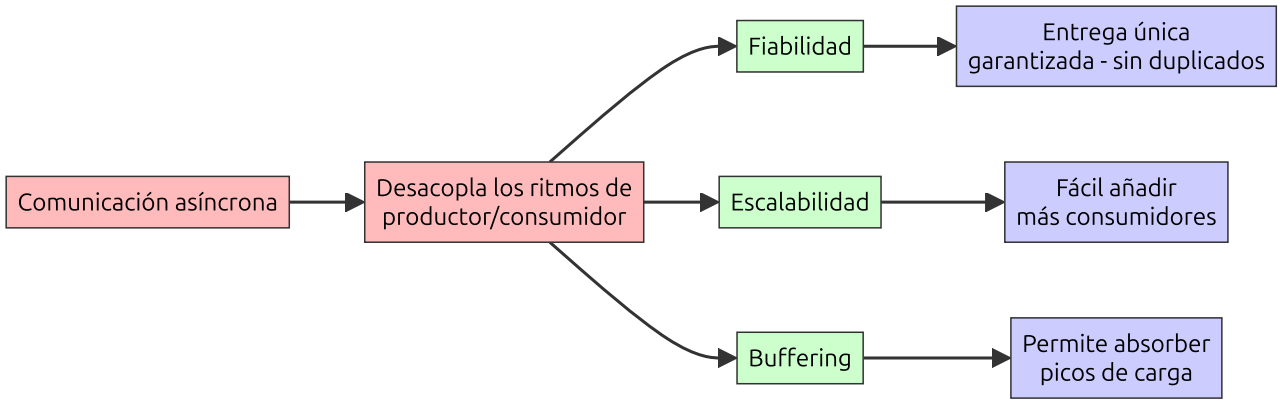
Componentes basicos:

- Mensaje: el objeto enviado por el productor y procesado por el consumidor
- Cola: buffer temporal que almacena los mensajes, normalmente utilizando un esquema FIFO

Ejemplos:

- Procesamiento de pedidos de comercio electronico: permite absorber picos de carga, y garantizr que los mensajes solo se consuman una vez
- Transacciones fimancieras y procesamiento de pagos
- Proteccion de datos altamente sensibles en reposo y en transito: colas de mensajes con cifrado de extremo a extremo seria una gran eleccion
- Procesos de larga duracion y trabajos en segundo plano: envio de correos electronicos, escalado de imagenes, escaneado de archivos, calculos matematicos intensivos, etc.

Beneficios



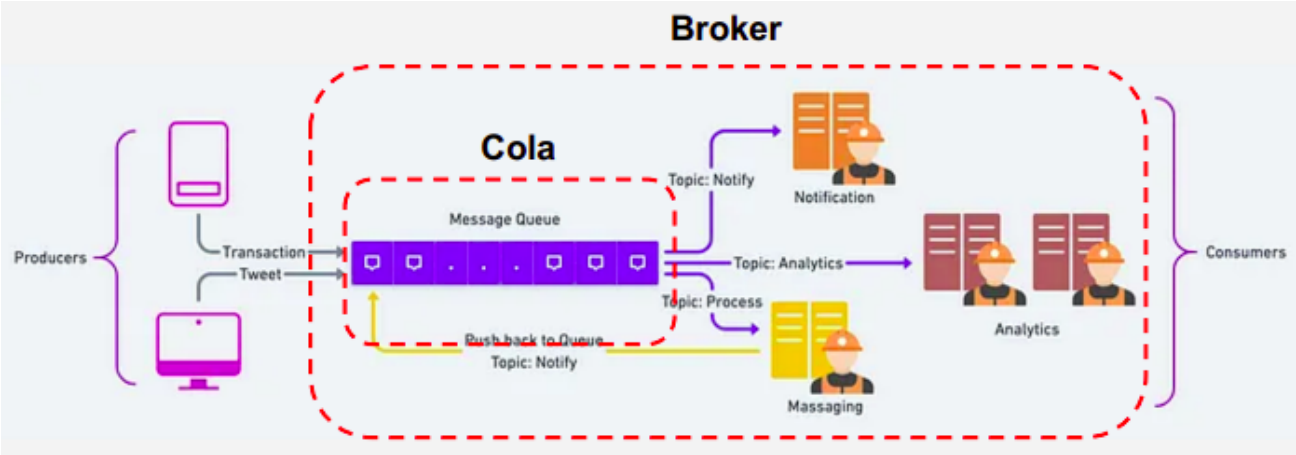

Caracteristicas

- Envio o retardo planificado
- Entrega garantizada: Las colas de mensajes permiten almacenar multiples copias de los mensajes para proporcionar redundancia y alta disponibilidad y reenviarlos en caso de un fallo de comunicacion y asegurar asi su entrega
- Colass Dead-letter (Colas de entrega fallidas)
- Mensajes de la muerte (poison-pill): mensajes especiales que se utilizan para señalizar a un consumidor que debe acabar su trabajo y no esperar nuevas entregas, similar a cerrar un socket en el modelo cliente/servidor

Estandares y Protocolos

- Nivel de aplicacion TCP/IP:
  - Advanced Message Queuing Protocol (AMQP)
  - Streaming Text Oriented Messaging Protocol (STOMP)
- Nivel de transporte TCP/IP:
  - Message Queue Telemetry Transport (MQTT)

El broker de mensajes es un middleware encargado de la recepcion, enrutado y gestion de los mensajes
Las colas de mensajes son una parte del broker y se encarga de la persistencia de estos

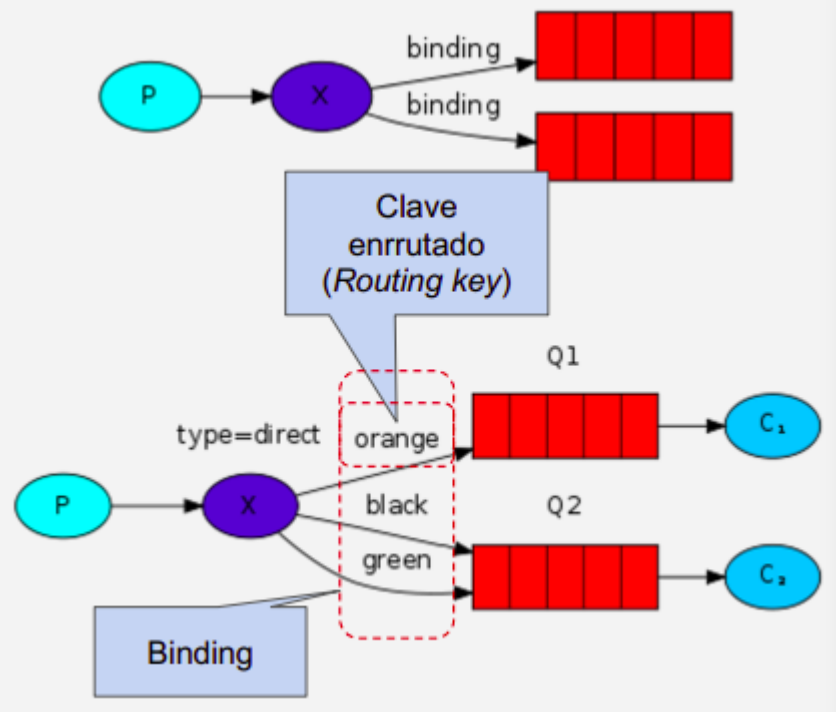



**RabbitMQ**

Uno de los brokers mas utilizados
Compuesto de:

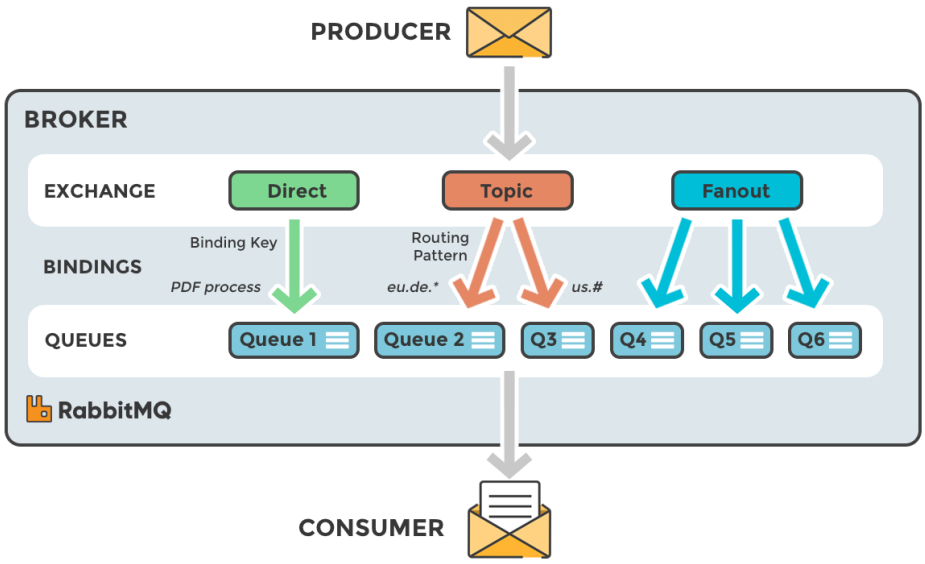
- Productores: clientes que crean los mensajes
- Consumidores: reciben y procesan los mensajes
- Colas: almacenan los mensajes

- Exchange: enrutan/descartan los mensajes a la cola adecuada Broker = Exchanges + Colas
  La vinculacion entre un exchange y una cola se llama binding (es una relacion N:N)




Tipos de exchanges:

- **Directos**: los exchanges directos envian los mensajes a las colas cuya clave coincide exactamente con la clave de enrutamiento, que el productor añade a la cabecera del mensaje.
  Ejemplo: un mensaje con clave pdf_log se envia al exchange pdf_events. El broker analizara sus cabeceras y lo enruta a la cola con la misma clave
  Si la clave de enrutamiento del mensaje no coincide con la clave de ninguna cola, el mensaje se descarta
- **Fanout**: un exchange fanout copia los mensajes recibidos a todas las colas vinculadas, independientemente de las claves de enrutamiento (completamente ignoradas)
  Pueden ser utiles cuando el mismo mensaje necesita ser enviado a una o mas colas consumidores que pueden procesar el mismo mensaje de diferentes maneras. Por ejemplo un chat por canales (Twitch), o actualizaciones deportivas o meteorologicas que deben enviarse a cada dispositivo movil conectado
- **Topic**: lista de plabras delimitadas por puntos y expresiones regulares



Tipos de arquitecturas/patrones

- Patron Punto-a-punto:
  - Relacion 1:1 entre cola y aplicacion
  - Puede haber mas de un consumidor escuchando en una cola, pero solo uno de ellos podra recibir el mensaje
    Ejemplo: enviar un correo al usuario al comprar un producto. La app utiliza una cola en la que coloca el correo a enviar, que se procesa por el primer consumidor disponible

## Cliente (Productor de mensajes)

In [1]:
```
import pika

connection_parameters = pika.ConnectionParameters('localhost')

connection = pika.BlockingConnection(connection_parameters)

channel = connection.channel()

channel.queue_declare(queue='letterbox')

message = "Hello this is my first message"

channel.basic_publish(exchange='', routing_key='letterbox', body=message)

print(f"sent message: {message}")

connection.close()
```

```
sent message: Hello this is my first message
```

## Servidor (Consumidor de mensajes)

Se modifica el codigo para que tenga un manejador de sigint

In [2]:
```
import pika

def on_message_received(ch, method, properties, body):
    print(f"received new message: {body}")

connection_parameters = pika.ConnectionParameters('localhost')

connection = pika.BlockingConnection(connection_parameters)

channel = connection.channel()

channel.queue_declare(queue='letterbox')

channel.basic_consume(queue='letterbox', auto_ack=True,
    on_message_callback=on_message_received)

print("Starting Consuming")

try:
    channel.start_consuming()
except KeyboardInterrupt:
    print("Interrupted. Closing connection...")
    channel.stop_consuming()
finally:
```

```
    connection.close()
    print("Connection closed.")
```

```
Starting Consuming
received new message: b'Hello this is my first message'
Interrupted. Closing connection...
Connection closed.
```

- Colas de trabajo:
  - Permiten que un cliente siga pudiendo atender acciones de un usuario mientras se realiza una tarea larga o intensiva en background
  - Util cuando no es posible realizar una tarea mientras se atiende a una peticion HTTP
  - Confirmacion de mensajes:
    - Un ACK es por el consumidor para indicar al exchange que un mensaje ha sido recibido, para borrarlo
    - Si un consumidor muere sin enviar el ACK, RabbitMQ volvera a poner en cola el mensaje
    - Se aplica un tiempo de espera en la confirmacion por parte del consumidor. Esto ayuda a detectar consumidores con errores.
  - Envio justo:
    - En una situacion con carga desigual, se activa el envio justo
    - Esto indica a RabbitMQ que no proporcione un nuevo mensaje a un trabajador hasta que este no haya confirmado el anterior

## Cliente (Productor de mensajes)

Se muestra el codigo, pero para el ejemplo se ha modificado para que se pueda usar en cuadernos jupyter

```
import pika
import time
import random

connection_parameters = pika.ConnectionParameters('localhost')

connection = pika.BlockingConnection(connection_parameters)

channel = connection.channel()

channel.queue_declare(queue='letterbox')

messageId = 1

while(True):
    message = f"Sending Message Id: {messageId}"

    channel.basic_publish(exchange='', routing_key='letterbox',
body=message)

    print(f"sent message: {message}")

    time.sleep(random.randint(1, 4))

    messageId+=1
```

## Servidor (Consumidor de mensajes)

```python
import pika
import time
import random

# Se hace el ack manual al acabar el procesamiento
def on_message_received(ch, method, properties, body):
    processing_time = random.randint(1, 6)
    print(f'received: "{body}", will take {processing_time} to process')
    time.sleep(processing_time)
    ch.basic_ack(delivery_tag=method.delivery_tag)
    print(f'finished processing and acknowledged message')

connection_parameters = pika.ConnectionParameters('localhost')

connection = pika.BlockingConnection(connection_parameters)

channel = connection.channel()

channel.queue_declare(queue='letterbox')

# Fair dispatching
# This will make sure that the server will not send a new message to a
consumer until it has acknowledged the previous one
channel.basic_qos(prefetch_count=1)

channel.basic_consume(queue='letterbox',
on_message_callback=on_message_received)

print('Starting Consuming')

channel.start_consuming()
```

**Ejemplo**:

In [10]:
```python
import multiprocessing
import pika
import time
import random
import signal
import sys

def producer():
    connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
    channel = connection.channel()
    channel.queue_declare(queue='letterbox')
    print("Producer started")

    messageId = 1
    try:
        while True:
            message = f"Sending Message Id: {messageId}"
            channel.basic_publish(exchange='', routing_key='letterbox', body=message)
            print(f"Producer sent message: {message}\n")
            time.sleep(random.randint(1, 4))
            messageId += 1
    except KeyboardInterrupt:
        print("Producer interrupted. Closing connection...")
    finally:
        connection.close()

def consumer(consumer_id):
```

```
    connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
    channel = connection.channel()
    channel.queue_declare(queue='letterbox')
    print(f"Consumer {consumer_id} started")

    def on_message_received(ch, method, properties, body):
        processing_time = random.randint(1, 6)
        print(f"Consumer {consumer_id} received: '{body.decode()}', will take {proces
        time.sleep(processing_time)
        ch.basic_ack(delivery_tag=method.delivery_tag)
        print(f"Consumer {consumer_id} finished processing and acknowledged message")

    try:
        channel.basic_qos(prefetch_count=1)
        channel.basic_consume(queue='letterbox', on_message_callback=on_message_recei
        print(f"Consumer {consumer_id} started consuming")
        channel.start_consuming()
    except KeyboardInterrupt:
        print(f"Consumer {consumer_id} interrupted. Closing connection...")
    finally:
        try:
            channel.stop_consuming()
        except:
            pass
        connection.close()

if __name__ == "__main__":
    # Create processes for producer and consumers
    producer_process = multiprocessing.Process(target=producer)
    consumer1_process = multiprocessing.Process(target=consumer, args=(1,))
    consumer2_process = multiprocessing.Process(target=consumer, args=(2,))

    # Start processes
    producer_process.start()
    consumer1_process.start()
    consumer2_process.start()

    def signal_handler(sig, frame):
        print("Main process interrupted. Terminating all processes...")
        producer_process.terminate()
        consumer1_process.terminate()
        consumer2_process.terminate()
        producer_process.join()
        consumer1_process.join()
        consumer2_process.join()
        print("All processes terminated.")

    # Handle SIGINT (Ctrl+C) to terminate processes gracefully
    signal.signal(signal.SIGINT, signal_handler)

    try:
        producer_process.join()
        consumer1_process.join()
        consumer2_process.join()
    except KeyboardInterrupt:
        signal_handler(None, None)
```

```
Producer started
Producer sent message: Sending Message Id: 1

Consumer 1 started
Consumer 1 started consuming
Consumer 1 received: 'Sending Message Id: 1', will take 2s to process
Consumer 2 started
Consumer 2 started consuming
Consumer 1 finished processing and acknowledged message
Producer sent message: Sending Message Id: 2
Consumer 1 received: 'Sending Message Id: 2', will take 4s to process

Producer sent message: Sending Message Id: 3
Consumer 2 received: 'Sending Message Id: 3', will take 2s to process

Consumer 1 finished processing and acknowledged message
Producer sent message: Sending Message Id: 4
Consumer 1 received: 'Sending Message Id: 4', will take 6s to processConsumer 2 finish
ed processing and acknowledged message


Producer sent message: Sending Message Id: 5
Consumer 2 received: 'Sending Message Id: 5', will take 5s to process

Consumer 1 finished processing and acknowledged message
Producer sent message: Sending Message Id: 6
Consumer 1 received: 'Sending Message Id: 6', will take 5s to process

Consumer 2 finished processing and acknowledged message
Producer sent message: Sending Message Id: 7
Consumer 2 received: 'Sending Message Id: 7', will take 3s to process

Consumer 1 finished processing and acknowledged message
Consumer 2 finished processing and acknowledged message
Producer sent message: Sending Message Id: 8
Consumer 1 received: 'Sending Message Id: 8', will take 5s to process

Producer sent message: Sending Message Id: 9
Consumer 2 received: 'Sending Message Id: 9', will take 5s to process

Consumer 1 finished processing and acknowledged message
Producer sent message: Sending Message Id: 10
Consumer 1 received: 'Sending Message Id: 10', will take 5s to process

Producer sent message: Sending Message Id: 11

Consumer 2 finished processing and acknowledged message
Consumer 2 received: 'Sending Message Id: 11', will take 2s to process
Producer sent message: Sending Message Id: 12

Consumer 2 finished processing and acknowledged message
Consumer 2 received: 'Sending Message Id: 12', will take 3s to process
Producer sent message: Sending Message Id: 13

Consumer 1 finished processing and acknowledged message
Consumer 1 received: 'Sending Message Id: 13', will take 1s to process
Producer sent message: Sending Message Id: 14

Consumer 1 finished processing and acknowledged message
Consumer 1 received: 'Sending Message Id: 14', will take 1s to process
Producer sent message: Sending Message Id: 15
```

```
Main process interrupted. Terminating all processes...
All processes terminated.
```

- Patron Publicador/suscriptor (o Productor/consumidor): Hay tres tipos: fanout, routing y topics:
  - Fanout: hara broadcast de los mensajes a todos los consumidores suscritos al exchange (Modelo Pub/sub normal)

## Cliente (Productor de mensajes)

In [23]:
```python
import pika
from pika.exchange_type import ExchangeType

connection_parameters = pika.ConnectionParameters('localhost')

connection = pika.BlockingConnection(connection_parameters)

channel = connection.channel()

channel.exchange_declare(exchange='pubsub', exchange_type=ExchangeType.fanout)

message = "Hello I want to broadcast this message"

channel.basic_publish(exchange='pubsub', routing_key='', body=message)

print(f"sent message: {message}")

connection.close()
```

```
sent message: Hello I want to broadcast this message
```

## Servidor (Consumidor de mensajes)

Se muestra el codigo, pero para el ejemplo se ha modificado para que se pueda usar en cuadernos jupyter

```python
import pika

def on_message_received(ch, method, properties, body):
    print(f"firstconsumer - received new message: {body}")

connection_parameters = pika.ConnectionParameters('localhost')

connection = pika.BlockingConnection(connection_parameters)

channel = connection.channel()

channel.exchange_declare(exchange='pubsub', exchange_type='fanout')

queue = channel.queue_declare(queue='', exclusive=True)

channel.queue_bind(exchange='pubsub', queue=queue.method.queue)

channel.basic_consume(queue=queue.method.queue, auto_ack=True,
    on_message_callback=on_message_received)

print("Starting Consuming")

channel.start_consuming()
```

In [ ]:
```python
import pika
import threading

def start_consumer(consumer_name):
    def callback(ch, method, properties, body):
        print(f"{consumer_name} received: {body.decode()}")

    connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
    channel = connection.channel()
    channel.exchange_declare(exchange='pubsub', exchange_type='fanout')
    queue = channel.queue_declare(queue='', exclusive=True)
    channel.queue_bind(exchange='pubsub', queue=queue.method.queue)
    channel.basic_consume(queue=queue.method.queue, on_message_callback=callback, aut
    print(f"{consumer_name} waiting for messages...")
    channel.start_consuming()

# Lanzar dos hilos consumidores
threading.Thread(target=start_consumer, args=('Consumer 1',), daemon=True).start()
threading.Thread(target=start_consumer, args=('Consumer 2',), daemon=True).start()
```

```
Consumer 2 waiting for messages...
Consumer 1 waiting for messages...
Consumer 2 received: Hello I want to broadcast this message
Consumer 1 received: Hello I want to broadcast this message
```

- Routing: para que el mensaje lo reciba una cola, esa cola tiene que tener exactamente el mismo routing_key que el routing_key del mensaje que se manda al exchange

## Cliente (Productor de mensajes)

In [25]:
```python
import pika
from pika.exchange_type import ExchangeType

connection_parameters = pika.ConnectionParameters('localhost')

connection = pika.BlockingConnection(connection_parameters)

channel = connection.channel()

channel.exchange_declare(exchange='routing', exchange_type=ExchangeType.direct)

message = 'This message needs to be routed by both services'

# Se especifica la clave de enrutamiento (en este caso, "both")
channel.basic_publish(exchange='routing', routing_key='both', body=message)

print(f'sent message: {message}')

message = 'This message needs to be routed by service analytics'

channel.basic_publish(exchange='routing', routing_key='analyticsonly', body=message)

print(f'sent message: {message}')

message = 'This message needs to be routed by service payments'

channel.basic_publish(exchange='routing', routing_key='paymentsonly', body=message)

print(f'sent message: {message}')

connection.close()
```

```
sent message: This message needs to be routed by both services
sent message: This message needs to be routed by service analytics
sent message: This message needs to be routed by service payments
```

## Servidor (Consumidor de mensajes)

Para este ejemplo se tienen dos servicios (simulados): el servicio analytics y el servicio payments. Lo unico que difiere de cada uno es la routing_key personalizada que tienen. Para el servicio analytics se tiene la routing_key 'analytics_only' y para el servicio payments la routing_key 'payments_only'. Para ambos servicios se tiene la clave compartida 'both'

In [ ]:

```python
import threading
import pika
from pika.exchange_type import ExchangeType

def start_analytics_consumer():
    def on_message_received(ch, method, properties, body):
        print(f'Analytics - received new message: {body}')

    connection_parameters = pika.ConnectionParameters('localhost')
    connection = pika.BlockingConnection(connection_parameters)
    channel = connection.channel()
    channel.exchange_declare(exchange='routing', exchange_type=ExchangeType.direct)
    queue = channel.queue_declare(queue='', exclusive=True)
    channel.queue_bind(exchange='routing', queue=queue.method.queue, routing_key='ana
    channel.queue_bind(exchange='routing', queue=queue.method.queue, routing_key='bot
    channel.basic_consume(queue=queue.method.queue, auto_ack=True, on_message_callbac

    print('Analytics Starting Consuming')
    channel.start_consuming()

def start_payments_consumer():
    def on_message_received(ch, method, properties, body):
        print(f'Payments - received new message: {body}')

    connection_parameters = pika.ConnectionParameters('localhost')
    connection = pika.BlockingConnection(connection_parameters)
    channel = connection.channel()
    channel.exchange_declare(exchange='routing', exchange_type=ExchangeType.direct)
    queue = channel.queue_declare(queue='', exclusive=True)
    channel.queue_bind(exchange='routing', queue=queue.method.queue, routing_key='pay
    channel.queue_bind(exchange='routing', queue=queue.method.queue, routing_key='bot
    channel.basic_consume(queue=queue.method.queue, auto_ack=True, on_message_callbac

    print('Payments Starting Consuming')
    channel.start_consuming()

# Lanzar ambos consumidores en hilos separados
analytics_thread = threading.Thread(target=start_analytics_consumer)
payments_thread = threading.Thread(target=start_payments_consumer)

analytics_thread.start()
payments_thread.start()
```

```
Analytics Starting Consuming
Payments Starting Consuming
Payments - received new message: b'This message needs to be routed by both services'
Analytics - received new message: b'This message needs to be routed by both services'
Analytics - received new message: b'This message needs to be routed by service analyti
cs'
Payments - received new message: b'This message needs to be routed by service payment
s'
```

- Topic: mandara el mensaje a las colas suscritas que ademas su routing_key coincida con la secuencia de letras seguidas por puntos (.) y hashtags (#)
  Ejemplo:
  - **users.europe.***: quiere decir que el routing_key tiene que coincidir con users.europe y da igual lo que haya despues que coincidira
  - ***.europe.purchases**: quiere decir que el routing_key tiene que acabar en europe.purchases, dando igual como empiece, siempre que se cumpla eso va a coincidir
  - **user.#**: quiere decir que el routing_key tiene que empezar por user, pero da igual como acabe siempre que se cumpla eso va a coincidir

## Cliente (Productor de mensajes)

In [27]:
```
import pika
from pika.exchange_type import ExchangeType

connection_parameters = pika.ConnectionParameters('localhost')

connection = pika.BlockingConnection(connection_parameters)

channel = connection.channel()

channel.exchange_declare(exchange='topic', exchange_type=ExchangeType.topic)

user_payments_message = 'A european user paid for something'

channel.basic_publish(exchange='topic', routing_key='user.europe.payments', body=user

print(f'sent message: {user_payments_message}')

business_order_message = 'A european business ordered goods'

channel.basic_publish(exchange='topic', routing_key='business.europe.order', body=bus

print(f'sent message: {business_order_message}')

connection.close()
```

```
sent message: A european user paid for something
sent message: A european business ordered goods
```

## Servidor (Consumidor de mensajes)

Para este ejemplo se definen 3 servicios (simulados):

- Servicio Analytics: tiene como routing_key: *.europe.*
- Servicio Payments: tiene como routing_key: #.payments
- Servicio User: tiene como routing_key: user.#

In [ ]:

```
import threading
import pika
from pika.exchange_type import ExchangeType

def start_analytics_consumer():
    def on_message_received(ch, method, properties, body):
        print(f'Analytics - received new message: {body}')

    connection_parameters = pika.ConnectionParameters('localhost')
    connection = pika.BlockingConnection(connection_parameters)
    channel = connection.channel()
    channel.exchange_declare(exchange='topic', exchange_type=ExchangeType.topic)
    queue = channel.queue_declare(queue='', exclusive=True)
    channel.queue_bind(exchange='topic', queue=queue.method.queue, routing_key='*.eur
    channel.basic_consume(queue=queue.method.queue, auto_ack=True, on_message_callbac

    print('Analytics Starting Consuming')
    channel.start_consuming()

def start_payments_consumer():
    def on_message_received(ch, method, properties, body):
        print(f'Payments - received new message: {body}')

    connection_parameters = pika.ConnectionParameters('localhost')
    connection = pika.BlockingConnection(connection_parameters)
    channel = connection.channel()
    channel.exchange_declare(exchange='topic', exchange_type=ExchangeType.topic)
    queue = channel.queue_declare(queue='', exclusive=True)
    channel.queue_bind(exchange='topic', queue=queue.method.queue, routing_key='#.pay
    channel.basic_consume(queue=queue.method.queue, auto_ack=True, on_message_callbac

    print('Payments Starting Consuming')
    channel.start_consuming()

def start_user_consumer():
    def on_message_received(ch, method, properties, body):
        print(f'User - received new message: {body}')

    connection_parameters = pika.ConnectionParameters('localhost')
    connection = pika.BlockingConnection(connection_parameters)
    channel = connection.channel()
    channel.exchange_declare(exchange='topic', exchange_type=ExchangeType.topic)
    queue = channel.queue_declare(queue='', exclusive=True)
    channel.queue_bind(exchange='topic', queue=queue.method.queue, routing_key='user.
    channel.basic_consume(queue=queue.method.queue, auto_ack=True, on_message_callbac

    print('User Starting Consuming')
    channel.start_consuming()

# Lanzar los tres consumidores en hilos separados
analytics_thread = threading.Thread(target=start_analytics_consumer)
payments_thread = threading.Thread(target=start_payments_consumer)
user_thread = threading.Thread(target=start_user_consumer)

analytics_thread.start()
payments_thread.start()
user_thread.start()
```

```
User Starting Consuming
Payments Starting Consuming
Analytics Starting Consuming
Payments - received new message: b'A european user paid for something'
Analytics - received new message: b'A european user paid for something'
User - received new message: b'A european user paid for something'
Analytics - received new message: b'A european business ordered goods'
```

- Patron Peticion/respuesta:
  - Permite la comunicacion bidireccional entre productor y consumidor
  - Ejemplo:
    1. Cuando se lanza un cliente crea una cola exclusiva para sus respuestas
    2. El cliente manda un mensaje con dos atributos ( `reply_to` ), establecido en la cola de respuesta y un valor unico para cada mensaje ( `Correlation_id` )
    3. El servidor recoge el mensaje, lo procesa y devuelve la respuesta utilizando la cola indicada en el campo `reply_to`
    4. El cliente espera en la cola de respuestas
    5. Cuando aparece un mensaje, comprueba el `correlation_id` y si coinciden, entrega el mensaje

## Cliente (Productor de mensajes, pero tambien consumidor por el patron que estamos usando)

Se hacen unas modificaciones y se unen los dos codigos usando hilos para que se pueda ejecutar usando cuadernos Jupyter

```
import pika
import uuid

def on_reply_message_received(ch, method, properties, body):
    print(f"reply recieved: {body}")

connection_parameters = pika.ConnectionParameters('localhost')

connection = pika.BlockingConnection(connection_parameters)

channel = connection.channel()

reply_queue = channel.queue_declare(queue='', exclusive=True)

channel.basic_consume(queue=reply_queue.method.queue, auto_ack=True,
    on_message_callback=on_reply_message_received)

channel.queue_declare(queue='request-queue')

cor_id = str(uuid.uuid4())
print(f"Sending Request: {cor_id}")

channel.basic_publish('', routing_key='request-queue',
properties=pika.BasicProperties(
    reply_to=reply_queue.method.queue,
    correlation_id=cor_id
), body='Can I request a reply?')

print("Starting Client")

channel.start_consuming()
```

## Servidor (igual que cliente pero mas sencillo)

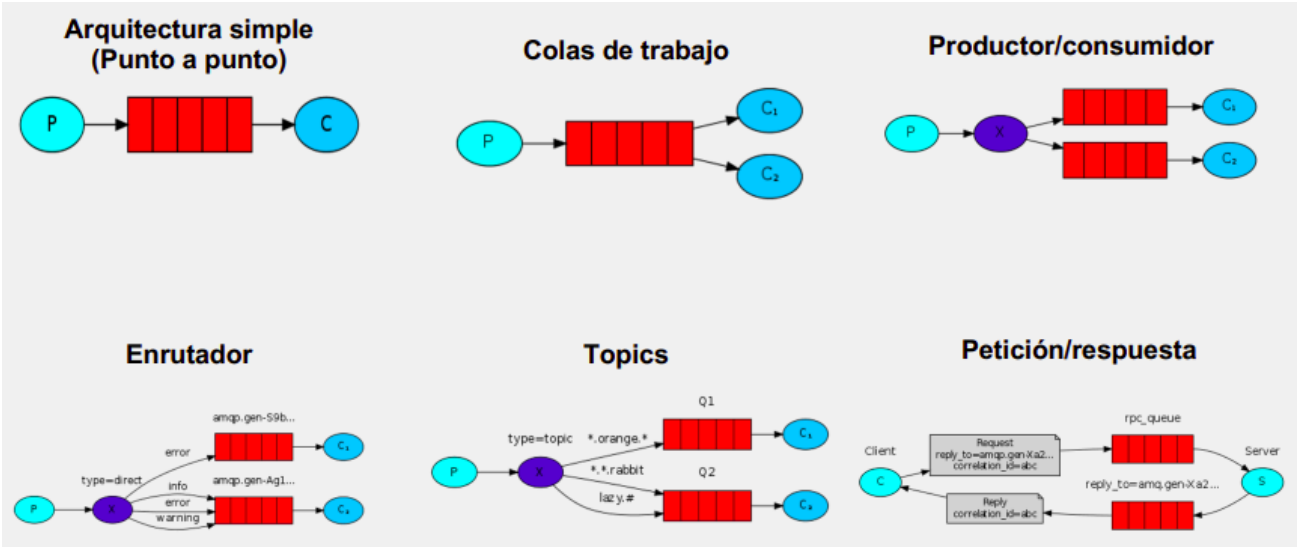
```
import pika

def on_request_message_received(ch, method, properties, body):
    print(f"Received Request: {properties.correlation_id}")
    ch.basic_publish('', routing_key=properties.reply_to, body=f'Hey its
your reply to {properties.correlation_id}')

connection_parameters = pika.ConnectionParameters('localhost')

connection = pika.BlockingConnection(connection_parameters)

channel = connection.channel()

channel.queue_declare(queue='request-queue')

channel.basic_consume(queue='request-queue', auto_ack=True,
    on_message_callback=on_request_message_received)

print("Starting Server")

channel.start_consuming()
```

In [29]:

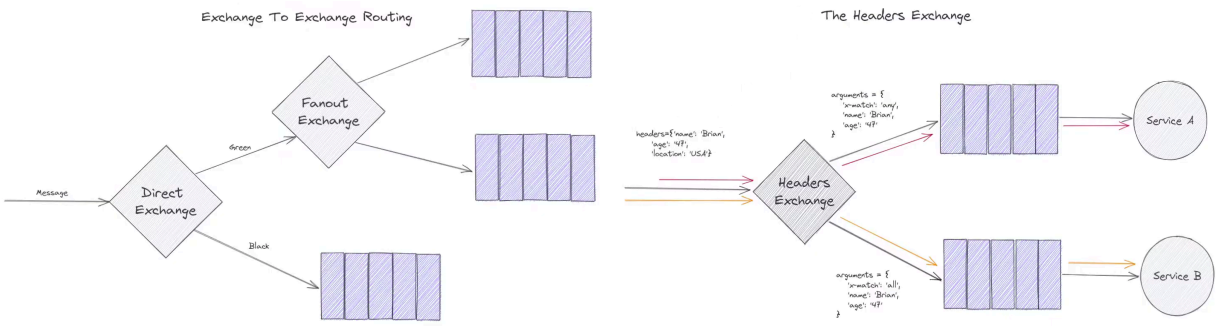
```
import threading
import pika
import uuid

def start_requester():
    def on_reply_message_received(ch, method, properties, body):
        print(f"[Requester] Reply received: {body.decode()}")

    connection_parameters = pika.ConnectionParameters('localhost')
    connection = pika.BlockingConnection(connection_parameters)
    channel = connection.channel()

    # Declaramos la cola de respuesta (exclusiva)
    reply_queue = channel.queue_declare(queue='', exclusive=True)

    channel.basic_consume(queue=reply_queue.method.queue, auto_ack=True,
                          on_message_callback=on_reply_message_received)

    channel.queue_declare(queue='request-queue')

    # Generamos una correlación única
    cor_id = str(uuid.uuid4())
    print(f"[Requester] Sending Request: {cor_id}")

    channel.basic_publish('', routing_key='request-queue', properties=pika.BasicPrope
        reply_to=reply_queue.method.queue,
        correlation_id=cor_id
    ), body='Can I request a reply?')

    print("[Requester] Waiting for reply...")
    channel.start_consuming()

def start_responder():
    def on_request_message_received(ch, method, properties, body):
        print(f"[Responder] Received request with correlation_id: {properties.correla
        response = f'Hey, it\'s your reply to {properties.correlation_id}'
        ch.basic_publish('', routing_key=properties.reply_to, body=response.encode())
```

```
        print(f"[Responder] Sent reply: {response}")

    connection_parameters = pika.ConnectionParameters('localhost')
    connection = pika.BlockingConnection(connection_parameters)
    channel = connection.channel()

    channel.queue_declare(queue='request-queue')

    channel.basic_consume(queue='request-queue', auto_ack=True,
                          on_message_callback=on_request_message_received)

    print("[Responder] Waiting for requests...")
    channel.start_consuming()

# Lanzar cliente y servidor en hilos separados
requester_thread = threading.Thread(target=start_requester)
responder_thread = threading.Thread(target=start_responder)

responder_thread.start()
requester_thread.start()
```

```
[Responder] Waiting for requests...
[Responder] Received request with correlation_id: 4e61711c-d96c-42f7-a95c-910b9f0b9898
[Responder] Sent reply: Hey, it's your reply to 4e61711c-d96c-42f7-a95c-910b9f0b9898
[Requester] Sending Request: bbffca05-606d-4be7-99e9-d30ce00e9ce2
[Requester] Waiting for reply...
[Responder] Received request with correlation_id: bbffca05-606d-4be7-99e9-d30ce00e9ce2
[Responder] Sent reply: Hey, it's your reply to bbffca05-606d-4be7-99e9-d30ce00e9ce2
[Requester] Reply received: Hey, it's your reply to bbffca05-606d-4be7-99e9-d30ce00e9c
e2
```




**Otros patrones (que derivan de los anteriormente vistos)**




**Escalado**

Una de las grandes ventajas de las colas de mensajes es que permiten aumentar el rendimiento de una aplicacion (escalado) facilmente:

- Incrementar el numero de consumidores, si la tasa de produccion de mensajes es mayor que la tasa de consumo
- Particionado a traves de topics, lo que ayuda a aumentar el grado de paralelismo:
  - Las colas pueden estar en distintos exchanges y hosts (cluster)